

CAPSTONE PROJECT (Project 4)

ShopNow: Production MERN Microservices Deployment on AWS EKS

GITHUB REPO: <https://github.com/anupam66kumar/shopNow.git>

An end-to-end DevOps Capstone project demonstrating the automated provisioning, configuration, containerization, deployment, and observability of an enterprise-grade, multi-tier e-commerce platform (**ShopNow**) on Amazon Elastic Kubernetes Service (EKS).

[ShopNow: Production MERN Microservices Deployment on AWS EKS](#)

1. [Project Overview](#)
 - 1.1. [Key Objectives](#)
2. [System Architecture](#)
3. [Repository Directory Structure](#)
4. [Tech Stack & Engineering Trade-Offs](#)
5. [CI/CD Automation Pipeline](#)
6. [Step-by-Step Deployment Runbook](#)
 - [Prerequisites](#)
 - 6.1. [Step 1: Infrastructure Deployment \(Terraform\)](#)
 - 6.2. [Step 2: Configuration Management \(Ansible\)](#)
 - 6.3. [Step 3: Application Rollout \(Kubernetes\)](#)
7. [Observability & Monitoring](#)
 - 7.1. [Accessing the Grafana Dashboard](#)
8. [Resilience & Autoscaling Testing](#)
 - 8.1. [Test 1: Pod Self-Healing Verification](#)
 - 8.2. [Test 2: Horizontal Pod Autoscaler \(HPA\) Verification](#)
9. [Root Cause Analysis \(RCA\) & Troubleshooting](#)
 - 9.1. [1. Docker Daemon Socket Permission Denied](#)
 - 9.2. [2. MongoDB Volume Stuck in Pending](#)
 - 9.3. [3. Backend Boot Failure & Mongoose Server Selection Timeout](#)
 - 9.4. [4. Cross-Subnet / Cross-Node Ingress Timeout](#)
 - 9.5. [5. Single-Page Application \(SPA\) Sub-path 404s](#)
10. [Resource Teardown](#)

1. Project Overview

The objective of this project is to build an automated, zero-touch deployment lifecycle for the **ShopNow** application. The solution replaces monolithic server setups with a resilient microservices architecture running on AWS EKS, managed via GitOps/Declarative CI/CD pipelines, and monitored with Prometheus and Grafana.

1.1 Key Objectives

- **Infrastructure as Code (IaC):** Modular Terraform code to provision a multi-AZ VPC, EKS Cluster, and EC2 Spot worker node groups.
- **Configuration Management:** Ansible playbooks for cluster controller bootstrap (NGINX Ingress Controller, Metrics Server, EBS CSI Driver).
- **Automated CI/CD:** Multi-stage Jenkins declarative pipelines triggered by GitHub Webhooks to test, build, push container images to private Amazon ECR, and roll out deployments.
- **Stateful Persistence:** Kubernetes StatefulSet for MongoDB backed by AWS EBS gp3 dynamic provisioning.
- **Ingress Routing & Path Rewriting:** Path-based micro-frontend routing directing external traffic to Storefront, Admin Dashboard, and REST API services through a single AWS Load Balancer.
- **Full-Stack Observability:** Telemetry and alert monitoring utilizing the kube-prometheus-stack with live Grafana dashboards.

2. System Architecture

3. Repository Directory Structure

```

shopNow/
├── .gitignore
├── README.md
├── admin/                                # Admin microservice (React frontend)
│   ├── Dockerfile                      # Multi-stage build with Nginx Alpine
│   ├── package.json
│   ├── package-lock.json
│   ├── nginx/
│   │   └── default.conf                # SPA routing and fallback configuration
│   ├── public/
│   └── src/
├── frontend/                            # Storefront microservice (React frontend)
│   ├── Dockerfile                      # Multi-stage build with Nginx Alpine
│   ├── package.json
│   ├── package-lock.json
│   ├── nginx/
│   │   └── default.conf                # Static asset serving configuration
│   ├── public/
│   └── src/
├── backend/                             # Core API microservice (Node.js/Express)
│   ├── Dockerfile                     # Production Alpine container with appuser
│   ├── package.json
│   ├── package-lock.json
│   ├── server.js                      # Express server, routes, and DB connector
│   └── routes/
├── ansible/                             # Configuration management playbooks
│   ├── ansible.cfg                    # Inventory path and run settings
│   ├── playbook.yml                  # Main orchestration playbook
│   ├── inventory/
│   │   └── hosts.ini                  # Local orchestrator host mapping
│   ├── roles/
│   │   └── k8s_prereqs/
│   │       └── tasks/
│   │           └── main.yml           # Installs Metrics Server, Ingress, gp3
│   └── main.yml                      # Infrastructure as Code definitions
├── terraform/                          # Remote S3 state backend configuration
│   ├── backend.tf                    # VPC, EKS cluster, and Node Group definitions
│   ├── main.tf                      # Cluster endpoint, Security Groups, kubeconfig
│   ├── outputs.tf                   # Input variable values
│   ├── terraform.tfvars             # Variable types and defaults
│   └── variables.tf                 # Kubernetes declarative manifests
├── k8s/
│   ├── namespace/
│   │   └── namespace.yaml            # shopnow-demo namespace
│   ├── database/
│   │   └── mongodb.yaml              # StatefulSet, Headless Service, gp3 VolumeClaim
│   ├── backend/
│   │   └── backend.yaml              # Deployment, ClusterIP Service, and HPA
│   ├── frontend/
│   │   └── frontend.yaml             # Deployment and ClusterIP Service
│   ├── admin/
│   │   └── admin.yaml                # Deployment and ClusterIP Service
│   ├── ingress/
│   │   └── ingress.yaml              # NGINX Ingress rules with regex rewriting
├── jenkins/                            # Declarative CI/CD pipeline scripts
│   ├── Jenkinsfile.terraform        # Stage 1: Terraform plan/apply
│   ├── Jenkinsfile.ansible          # Stage 2: Ansible configuration
│   └── Jenkinsfile.app               # Stage 3: Build, push to ECR, & rollout

```

4. Tech Stack & Engineering Trade-Offs

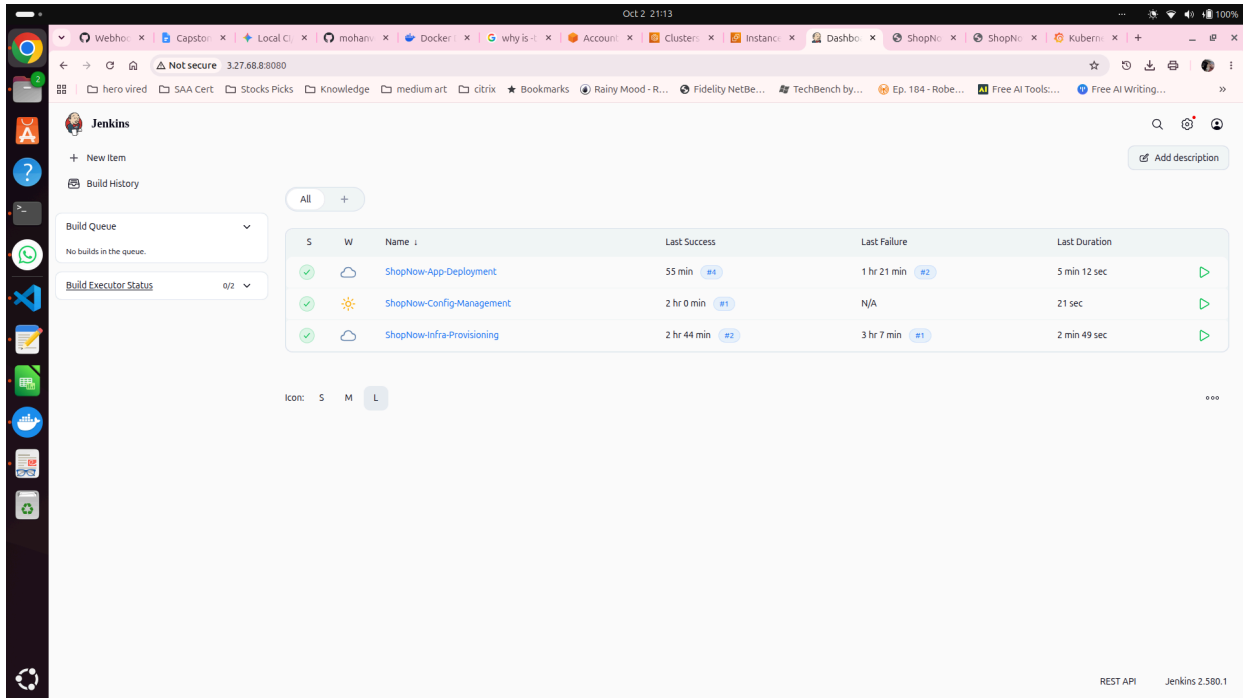
Component	Selected Technology	Alternative Considered	Engineering Rationale
Cloud Provider	AWS (Asia Pacific - Sydney)	GCP / Azure	Deep integration with EKS, AWS ECR, and fine-grained IAM Role policies for CSI drivers.
Orchestration	AWS EKS (v1.30)	Self-managed Kubernetes	Eliminates control-plane management overhead; native integration with AWS VPC CNI.
Worker Nodes	EC2 Spot Instances (t3.medium)	On-Demand Instances	Achieves up to 70% cost reduction during development while testing stateless failover resilience.
Node AMI	Amazon Linux 2023 (AL2023)	Amazon Linux 2 (AL2)	AL2 bootstrap scripts are deprecated in Kubernetes 1.30+. AL2023 uses standardized systemd initialization.
Stateful DB	StatefulSet + EBS CSI (gp3)	AWS RDS / DocumentDB	Demonstrates native Kubernetes volume persistence and dynamically provisioned PVC life-cycles.
Web Routing	NGINX Ingress Controller	AWS ALB Ingress Controller	Lightweight, supports rapid local regex rewriting rules (rewrite-target: /\$2), and avoids multi-ALB billing.
CI/CD	Jenkins Declarative Pipelines	GitHub Actions	Complete on-premise pipeline ownership, local Docker socket execution, and direct IAM node role integration.
Observability	Prometheus Operator + Grafana	AWS CloudWatch Container Insights	Open-source, vendor-neutral metric scraping with zero per-metric AWS CloudWatch ingestion fees.

5. CI/CD Automation Pipeline

The deployment process follows a 3-tier chained architecture:



[SCREENSHOT: Jenkins Pipelines]

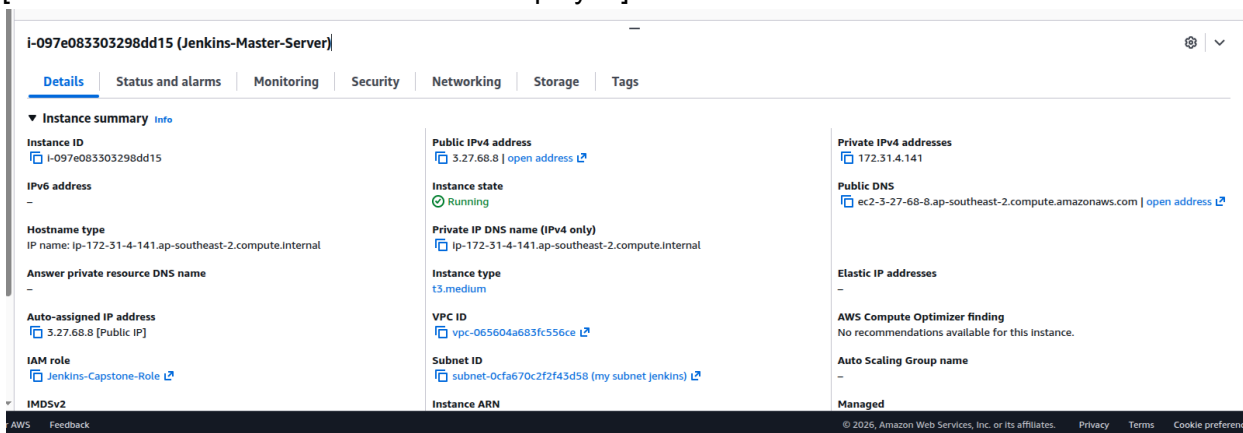


6. Step-by-Step Deployment Runbook

Prerequisites

- AWS CLI installed and configured with appropriate IAM Administrator credentials.
- Terraform v1.5+ and Ansible v2.15+ installed.
- Docker and kubectl matching Kubernetes v1.30.

[SCREENSHOT: Jenkins master EC2 Deployed]



6.1 Step 1: Infrastructure Deployment (Terraform)

1. Navigate to the terraform directory:

```
Bash
cd terraform
terraform init
terraform plan -var-file="terraform.tfvars"
```

2. Provision the AWS resources:

```
Bash
terraform apply -var-file="terraform.tfvars" -auto-approve
```

3. Update local cluster credentials:

```
Bash
aws eks update-kubeconfig --region ap-southeast-2 --name
project4-shopnow-eks
kubectl get nodes -o wide
```

6.2 Step 2: Configuration Management (Ansible)

1. Run the Ansible playbook to bootstrap cluster controllers:

```
Bash
cd ../ansible
ansible-playbook -i inventory/hosts.ini playbook.yml
```

2. Verify ingress controller and storage class readiness:

```
Bash
kubectl get sc
kubectl get pods -n ingress-nginx
kubectl get svc -n ingress-nginx
```

6.3 Step 3: Application Rollout (Kubernetes)

1. Create the application namespace:

```
Bash
kubectl apply -f k8s/namespace/
```

2. Deploy the MongoDB StatefulSet:

```
Bash
kubectl apply -f k8s/database/
kubectl rollout status statefulset/mongo -n shopnow-demo --timeout=180s
```

3. Initialize the database authentication credentials:

```
Bash
kubectl -n shopnow-demo exec -it mongo-0 -- mongosh --eval "
use admin;
db.createUser({
```

```
user: 'shopuser',  
pwd: 'ShopNowPass123',  
roles: [  
  { role: 'readWrite', db: 'shopnow' },  
  { role: 'dbAdmin', db: 'shopnow' }  
]  
));  
"
```

4. Deploy the microservices and ingress rules:

Bash

```
kubectl apply -f k8s/backend/  
kubectl apply -f k8s/frontend/  
kubectl apply -f k8s/admin/  
kubectl apply -f k8s/ingress/
```

5. Confirm rollout status across all deployments:

Bash

```
kubectl rollout status deployment/backend -n shopnow-demo --timeout=120s  
kubectl rollout status deployment/frontend -n shopnow-demo --timeout=120s  
kubectl rollout status deployment/admin -n shopnow-demo --timeout=120s
```

[SCREENSHOT: EKS Cluster Deployed]

Introducing the new cluster page.

You can find the cluster configurations in the Configuration tab, and navigate to other cluster resources from the side navigation. You can [switch back to the old cluster page](#) if you prefer.

Take a tour

project4-shopnow-eks



Connect

Actions

Monitor cluster

End of extended support for Kubernetes version (1.31) is November 26, 2026.

Upgrade

Cluster info

Status

Active

Kubernetes version

1.31

Support period

Extended support until November 26, 2026

Created

4 hours ago

Cluster ARN

arn:aws:eks:ap-southeast-2:951066974787:cluster/project4-shopnow-eks

API server endpoint

https://BCB8026D0650B6A66698B14D12461B25.gr7.ap-southeast-2.eks.amazonaws.com

Certificate authority

LS0LS1CRUdJTiBDRVJUSUJZQ0FURS0LS0tCk1JSURNVENDQWhtZD

OpenID Connect provider URL

https://oidc.eks.ap-southeast-2.amazonaws.com/id/BCB8026D0650B6A66698B14D12461B25

Cluster IAM role ARN

arn:aws:iam::951066974787:role/project4-shopnow-eks-cluster-2026100212361652230000001

[View in IAM](#)

Provider

EKS

Platform version

eks.71

Cluster health

0

Upgrade Insights

8

Node health issues

0

Capability issues

0

Cluster resources

Last updated: October 2, 2026, 21:43 (UTC+05:30)

Create

Data unavailable

Data unavailable

1

0

1

0

IAM access entries

3

Pod identities

0

OIDC identity providers

0

Configuration

Updated

Update history

Certificate authority

Cluster configuration

Manage

EKS Auto Mode

Disabled

Upgrade policy

Extended support

Authentication mode

EKS API and ConfigMap

Scaling tier

Standard [View usage](#)

Deletion protection

Disabled

ARC Zonal shift

Disabled

Encryption key

Customer managed

KMS key ID

60235d15-2c31-4e5c-9664-4cc775ccb238

Networking

Manage

VPC

vpc-04202176dafcd6fdc

Cluster IP address family

IPv4

Service IPv4 range

172.20.0.0/16

Subnets

subnet-0474c06095989b1b6

subnet-03d6fbb162459cc8d

subnet-013004d228e969ea1

Cluster security group

sg-0f5e3f0a13d9299dd

Additional security groups

sg-03a563c73e45a8d30

API server endpoint access

Public and private

Egress mode

AWS managed

Public access source allowlist

0.0.0.0/0 (open to all traffic)

Control plane configuration - new

Manage

Enabled

No

Configuration details

Container network observability

View network observability

Edit

[SCREENSHOT: EKS Nodes on EC2]

The screenshot displays the AWS Management Console for the 'ap-southeast-2' region. The 'Instances' page is active, showing a list of three EC2 instances. The instances are 'spot-worker-nodes' and 'Jenkins-Master-Server'. The 'Jenkins-Master-Server' instance is highlighted. The console also shows 'All instances' summary cards for alarm states, service health, and health events.

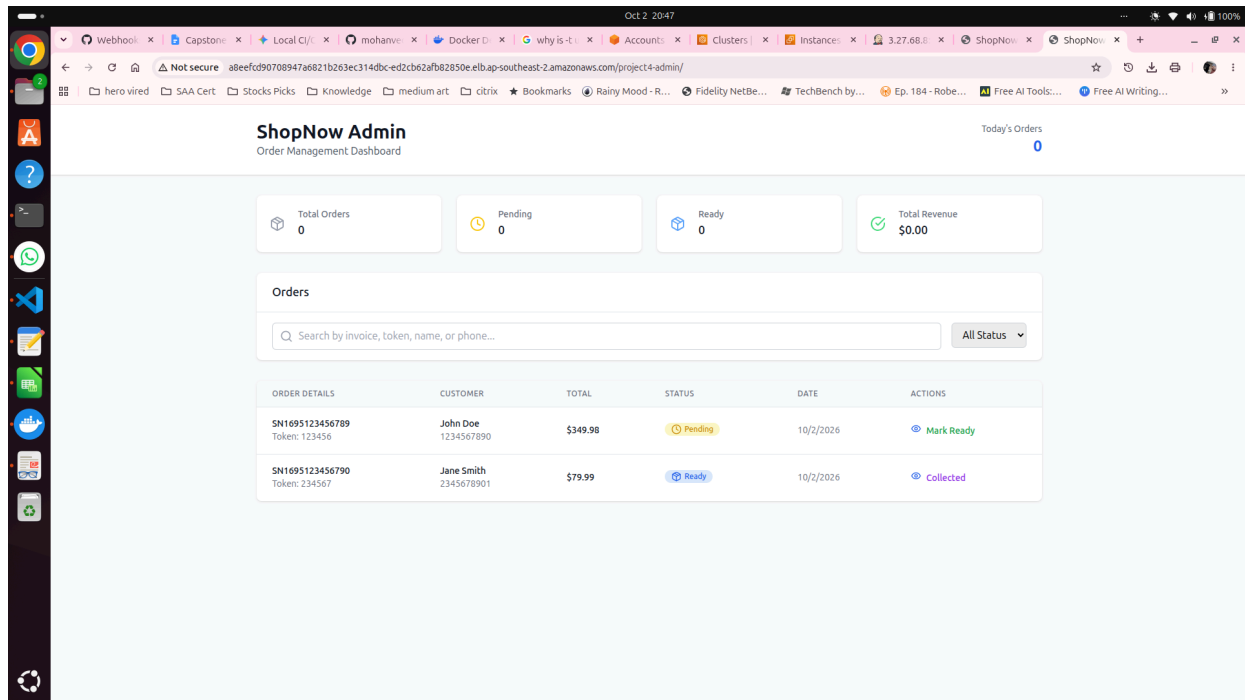
Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP
spot-worker-nodes	i-0699a18a315cdfc5	Running	t3.medium	3/3 checks passed	ap-southeast-2c	-	-	-
Jenkins-Master-Server	i-097e083503296dd15	Running	t3.medium	3/3 checks passed	ap-southeast-2a	ec2-3-27-68-8-ap-south...	3.27.68.8	-
spot-worker-nodes	i-0b122e915da85dca8	Running	t3.medium	3/3 checks passed	ap-southeast-2a	-	-	-

[SCREENSHOT: Customer Panel of ShopNow App]

The screenshot shows the ShopNow app's customer panel. It features a 'Welcome to ShopNow' banner with a 'Shop Now' button. Below the banner, there is a 'Products' section displaying four items: 'Coffee Maker', 'Running Shoes', 'Laptop Stand', and 'Organic Cotton T-Shirt'. Each product card includes an image, title, description, rating, price, and an 'Add to Cart' button.

Product	Price	Stock
Coffee Maker	\$79.99	25 in stock
Running Shoes	\$129.99	40 in stock
Laptop Stand	\$45.99	60 in stock
Organic Cotton T-Shirt	\$29.99	100 in stock

[SCREENSHOT: Admin Panel of ShopNow App]



7. Observability & Monitoring

The monitoring stack is deployed via Helm using the kube-prometheus-stack chart.

Bash

1. Add Prometheus Helm Repository

helm repo add prometheus-community

<https://prometheus-community.github.io/helm-charts>

helm repo update

2. Deploy the Stack with custom resource limits and LoadBalancer access

**helm install monitoring prometheus-community/kube-prometheus-stack **

**--namespace monitoring --create-namespace **

**--set grafana.service.type=LoadBalancer **

**--set grafana.adminPassword="AdminPassword123!" **

**--set grafana.resources.requests.cpu=100m **

--set grafana.resources.requests.memory=256Mi

7.1 Accessing the Grafana Dashboard

1. Retrieve the Load Balancer DNS:

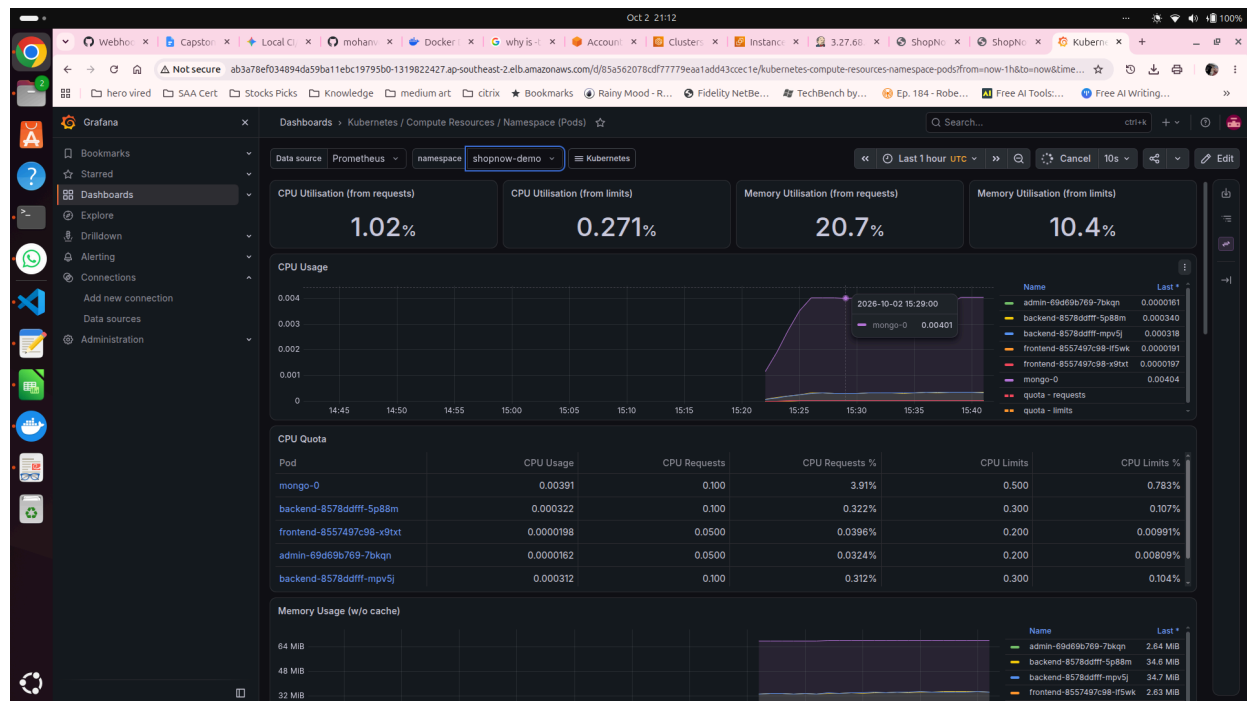
Bash

kubectl get svc monitoring-grafana -n monitoring -o

jsonpath='{.status.loadBalancer.ingress[0].hostname}'

- Open the URL in your browser:
 - Username:** admin
 - Password:** AdminPassword123!
- Select Dashboard: **Kubernetes / Compute Resources / Namespace (Pods)** → Select shopnow-demo to view real-time CPU, memory, network, and quota metrics.

[SCREENSHOT: Grafana Dashboard showing CPU and Memory Utilization for the nodes in Shop-Now-Demo Namespace using Prometheus as DataSource]



8. Resilience & Autoscaling Testing

8.1 Test 1: Pod Self-Healing Verification

Confirm that Kubernetes automatically reconciles desired state if a pod crashes or is terminated:

Bash

Delete an active backend pod

kubectl delete pod -n shopnow-demo -l app=backend --now | head -n 1

Watch replacement replica provision immediately

kubectl get pods -n shopnow-demo -l app=backend -w

8.2 Test 2: Horizontal Pod Autoscaler (HPA) Verification

Generate simulated HTTP traffic to trigger CPU thresholds (>75% utilization):

Bash

In Terminal 1: Watch the HPA state

kubectl get hpa backend-hpa -n shopnow-demo -w

In Terminal 2: Run a multi-threaded load generator

kubectl run load-generator --image=curlimages/curl --rm -it

--restart=Never -- /bin/sh -c '

for i in \$(seq 1 15); do

(while true; do

curl -s

http://backend-service.shopnow-demo.svc.cluster.local:5000/api/products >

/dev/null

curl -s http://backend-service.shopnow-demo.svc.cluster.local:5000/api/health >

/dev/null

done) &

done

wait

|

- **Observed Result:**

Replica count automatically scaled from 2 to 4 pods once CPU target spiked past 75%.
Upon stopping the load generator, replicas cooled down back to 2.

[SCREENSHOT: HPA and LoadTest]

```
ubuntu@ip-172-31-4-141:~$ # 1. Note the current pod names
kubectl get pods -n shopnow-demo -l app=backend

# 2. Delete one replica
kubectl delete pod -n shopnow-demo -l app=backend --now | head -n 1

# 3. Verify immediate reconstitution
kubectl get pods -n shopnow-demo -l app=backend -w
NAME READY STATUS RESTARTS AGE
backend-8578ddfff-5p88m 1/1 Running 0 53m
backend-8578ddfff-mpv5j 1/1 Running 0 53m
pod "backend-8578ddfff-5p88m" deleted from shopnow-demo namespace
NAME READY STATUS RESTARTS AGE
backend-8578ddfff-5p88m 1/1 Terminating 0 53m
backend-8578ddfff-6wt9d 0/1 ContainerCreating 0 1s
backend-8578ddfff-mpv5j 1/1 Terminating 0 53m
backend-8578ddfff-trf4c 0/1 ContainerCreating 0 1s
backend-8578ddfff-6wt9d 0/1 Running 0 1s
backend-8578ddfff-trf4c 0/1 Running 0 2s
backend-8578ddfff-mpv5j 0/1 Error 0 53m
backend-8578ddfff-5p88m 0/1 Error 0 54m
backend-8578ddfff-mpv5j 0/1 Error 0 53m
backend-8578ddfff-mpv5j 0/1 Error 0 53m
backend-8578ddfff-mpv5j 0/1 Error 0 53m
backend-8578ddfff-5p88m 0/1 Error 0 54m
backend-8578ddfff-5p88m 0/1 Error 0 54m
backend-8578ddfff-6wt9d 1/1 Running 0 20s
backend-8578ddfff-trf4c 1/1 Running 0 20s
^Cubuntu@ip-172-31-4-141:~$ # 1. Watch HPA status in one split/terminal window
kubectl get hpa backend-hpa -n shopnow-demo -w

# 2. Run a temporary load generator container to issue concurrent requests
kubectl run load-generator --image=busybox:1.28 --rm -it --restart=Never -- /bin/sh -c "while true; do wget -q -O- http://backend-service.shopnow-demo.svc.cluster.local:5000/api/health; done"
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
backend-hpa Deployment/backend cpu: 1%/75% 2 4 2 81m
```

```
ubuntu@ip-172-31-4-141:~$ # 1. Watch HPA status in one split/terminal window
kubectl get hpa backend-hpa -n shopnow-demo -w

# 2. Run a temporary load generator container to issue concurrent requests
kubectl run load-generator --image=busybox:1.28 --rm -lt --restart=Never -- /bin/sh -c "while true; do wget -q -O- http://backend-service.shopnow-demo.svc.cluster.local:5000/api/health; done"

NAME          REFERENCE          TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa   Deployment/backend  cpu: 1%/75%      2          4          2           81m

backend-hpa   Deployment/backend  cpu: 112%/75%    2          4          2           88m
backend-hpa   Deployment/backend  cpu: 264%/75%    2          4          3           88m
backend-hpa   Deployment/backend  cpu: 241%/75%    2          4          4           88m

ubuntu@ip-172-31-4-141:~$ kubectl get pods -n shopnow-demo -l app=backend
NAME                                READY   STATUS    RESTARTS   AGE
backend-8578ddfff-6wt9d             1/1     Running   0           10m
backend-8578ddfff-95gph             1/1     Running   0           82s
backend-8578ddfff-fmnltn            1/1     Running   0           97s
backend-8578ddfff-trf4c             1/1     Running   0           10m
ubuntu@ip-172-31-4-141:~$
```

9. Root Cause Analysis (RCA) & Troubleshooting

During the lifecycle of this deployment, several technical bottlenecks were diagnosed and resolved:

9.1. Docker Daemon Socket Permission Denied

- **Symptom:** Pipeline failure during docker build (permission denied while trying to connect to the docker API at `unix:///var/run/docker.sock`).
- **Root Cause:** The jenkins system user was not an active member of the host's docker UNIX group.
- **Resolution:** Executed `sudo usermod -aG docker jenkins`, set socket permissions with `sudo chmod 666 /var/run/docker.sock`, and restarted docker and jenkins services.

9.2. MongoDB Volume Stuck in Pending

- **Symptom:** Pod `mongo-0` remained in Pending indefinitely; PVC showed Pending.
- **Root Cause:** The EKS Spot worker node IAM role lacked AWS permissions to call EC2 volume attachment APIs.
- **Resolution:** Attached the AWS-managed policy `AmazonEBSCSIDriverPolicy` to the worker node IAM role and restarted `ebs-csi-controller`.

9.3. Backend Boot Failure & Mongoose Server Selection Timeout

- **Symptom:** Backend pods repeatedly entered `CrashLoopBackOff` (Exit Code 137).
- **Root Cause:**
 - The code expected the environment variable `MONGODB_URI`, while the manifest declared `MONGO_URI`. The app fell back to `localhost:27017` instead of `mongo:27017`.
 - The liveness probe was checking `/health`, while the Express route

was defined under `/api/health`.

- **Resolution:** Standardized `k8s/backend/backend.yaml` to export both `MONGODB_URI` and `MONGO_URI` pointing to `mongodb://mongo:27017/shopnow`, and updated container probes to query `/api/health`.

9.4. Cross-Subnet / Cross-Node Ingress Timeout

- **Symptom:** Ingress returned 504 Gateway Time-out when routing to pods located on different worker nodes (`connect()` failed 110: Operation timed out).
- **Root Cause:** The EKS Node Security Group lacked ingress rules allowing cross-node overlay traffic between worker instances residing in different subnets/availability zones.
- **Resolution:** Added a security group rule allowing all traffic (-1) where the source group was the Node Security Group itself, and permitted the VPC CIDR `10.0.0.0/16`.

9.5. Single-Page Application (SPA) Sub-path 404s

- **Symptom:** Accessing `/project4` yielded a blank white page; static assets under `/project4/static/js/...` threw 404s.
- **Root Cause:** React was built with `PUBLIC_URL=/project4`, but the ingress controller lacked regex path stripping.
- **Resolution:** Updated `k8s/ingress/ingress.yaml` to include regex capture groups path: `/project4(/|)(.*)` alongside annotation `nginx.ingress.kubernetes.io/rewrite-target: /2`.

10. Resource Teardown

To avoid incurring ongoing AWS charges after completing the evaluation:

1. Delete Kubernetes & Helm Resources:

```
Bash
```

```
helm uninstall monitoring -n monitoring
```

```
kubectl delete namespace shopnow-demo monitoring
```

2. Destroy AWS Infrastructure via Terraform:

```
Bash
```

```
cd terraform
```

```
terraform destroy -var-file="terraform.tfvars" -auto-approve
```

3. **Terminate Jenkins Master:** Stop or terminate the Jenkins EC2 instance from the AWS EC2 Management Console.